CRIPTA — Documento de Decisiones de Diseño

Oct 16, 2026 · @Cesar

CRIPTA es un juego de exploración de criptas por eventos: no hay rondas, solo un reloj virtual que ejecuta lo que toque en cada instante, y este documento explica por qué cada estructura de datos que lo sostiene fue una buena elección y qué otra habría funcionado.

Cómo leer este documento

Cada sección cubre un problema del §4 del enunciado y sigue el orden que pide el §8: el problema con las operaciones que el juego realmente ejecuta, la decisión, su complejidad, una alternativa que también habría servido con un pro y un contra de cada lado, y los límites donde la decisión dejaría de ser apropiada. Cuando varias piezas del código usan la misma estructura (por ejemplo, tres listas enlazadas), se agrupan y se discuten una sola vez.

Las restricciones que moldearon todas las decisiones vienen del enunciado:

- **§7:** ningún `dict` como índice o almacenamiento del modelo; `list` solo como almacenamiento contiguo de bajo nivel, y el resto con estructuras propias.
- **§2.3 y §4.1:** el tiempo es un entero virtual, no el reloj de pared, y el orden de ejecución debe ser determinista (mismo tiempo, desempata el número de secuencia).
- **§4.9:** nada de `list.sort()` ni `sorted()` en los ordenamientos evaluados.
- **§5 y §9:** el presupuesto de solicitudes de red es parte del contrato, y la defensa será con una cripta que no vimos, posiblemente mucho mayor.

Estado de la evidencia empírica

Todas las cifras de este documento salen de `python3 main.py --bench` (semilla 2026), ejecutado el 16 de octubre de 2026 en Linux aarch64 con Python 3.10.12, con el recolector de basura ajustado como en una partida. El proyecto exige Python 3.11+, así que falta repetir la corrida ahí. Cada valor es la mediana de varias repeticiones (el mínimo y el máximo quedan en `bench_final.csv`), y se mide también la alternativa descartada, como pide el §8. Los tiempos absolutos dependen de la máquina; lo que se compara es la razón

entre estructuras. **En cuatro casos la medición contradijo lo que se suponía al diseñar** (agenda, inventario, carga completa del guardado y bitácora en memoria), y cada sección lo dice.

1. Agenda de eventos del reloj virtual

La agenda es un **montículo binario (heap) de mínimos escrito a mano**, ordenado por (tiempo de ejecución, número de secuencia), donde cada evento recuerda su posición dentro del arreglo.

Problema. Todo lo que ocurre en CRIPTA es un evento en el futuro: cada acción de un enemigo, el cierre automático de una puerta, el rearme de una trampa, el fin de una poción o de una antorcha. El juego repite un ciclo sin parar: mirar cuál es el evento más próximo, sacarlo, ejecutarlo y, casi siempre, programar otro. Además hay dos operaciones menos frecuentes pero obligatorias: **cancelar** un evento que ya estaba programado (el pergamino deshace una puerta que iba a cerrarse sola, un enemigo muere con una acción pendiente) y **reprogramar** el tiempo restante de las acciones de un actor cuando su velocidad cambia (§2.3). Con varios cientos de eventos pendientes, la operación que se repite decenas de miles de veces es «dame el más próximo», y hacerla mal (recorrer todo en cada paso) multiplica el costo de cada ciclo por el tamaño de la agenda.

Por qué fue buena decisión.

- Insertar y sacar el mínimo cuestan O(log n); consultar el siguiente sin sacarlo cuesta O(1) (`heap[0]`), que es lo que necesita el ciclo para decidir si ya toca el turno del jugador.
- El desempate (§4.1) queda dentro de la propia comparación: a igual tiempo gana el número de secuencia más bajo, así que dos corridas con la misma semilla ejecutan los eventos en el mismo orden, requisito del `--replay`.
- Cada evento guarda su posición en el arreglo, lo que permite **quitar un evento cualquiera en O(log n)** sin recorrer la agenda. Sin ese dato, cancelar una puerta o reprogramar una acción costaría O(n) cada vez.
- Reprogramar no modifica el evento: lo reemplaza por uno nuevo con número de secuencia nuevo, como pide §2.3, y ambos movimientos quedan en el historial de retroceso (sección 3), así que deshacerlo es mover los mismos objetos en sentido contrario.

Complejidad.

Operación	Costo	Frecuencia en el juego
Programar un evento	O(log n)	Una o más por cada acción
Sacar el siguiente	O(log n)	Una por cada evento ejecutado
Ver el siguiente	O(1)	Una por evento, para decidir el turno
Cancelar un evento concreto	O(log n)	Ocasional: puertas, muertes, pergamino
Reajustar velocidad de un actor	O(n)	Solo al empezar o terminar un efecto de velocidad

El reajuste de velocidad es lineal a propósito: ocurre dos veces por poción, no por ciclo, y mantener un índice por actor para evitarlo habría añadido una segunda estructura que sincronizar sin ganar nada medible.

Alternativa que también habría funcionado: lista ordenada con inserción por búsqueda binaria.

	Lista ordenada (arreglo contiguo)	Heap con posición (elegido)
Pro	Ver y sacar el siguiente es O(1) si se mantiene en orden inverso; el código es más corto y la agenda queda siempre ordenada, lo que facilita depurar y mostrarla	Insertar y cancelar son O(log n) incluso con muchos eventos simultáneos
Contra	Cada inserción o cancelación desplaza elementos: O(n). Con varios cientos de eventos y criptas más grandes, el costo crece justo en la operación frecuente	Solo el mínimo está ordenado; para ver toda la agenda hay que extraer o copiar y ordenar, y hay que mantener el campo `posicion` sincronizado en cada intercambio

Para las agendas reales del juego (hasta 1 000 eventos pendientes) la lista ordenada habría dado resultados equivalentes y más simples; la medición de abajo muestra que incluso fue más rápida en el ciclo habitual. El heap se conserva por su costo O(log n) garantizado en la defensa (§9), no por haber ganado en velocidad.

Límites conocidos. Si la agenda nunca superara unas decenas de eventos, la diferencia con la lista ordenada sería imperceptible y el heap sobraría. Si se necesitara consultar la agenda por actor con frecuencia (por ejemplo, para mostrar «próxima acción de cada enemigo»), el barrido lineal por actor dejaría de ser aceptable y habría que añadir un índice por actor.

Evidencia empírica (`--bench`). Ambas estructuras sacan los eventos en el mismo orden (verificado con 2 000 eventos con empates). Tiempos con N eventos en la agenda:

Operación	N	Heap con posición	Lista ordenada
Programar N eventos	100	67.4 µs	75.1 µs
Programar N eventos	1 000	732 µs	1.08 ms
Programar N eventos	10 000	7.56 ms	20.15 ms
Sacar los N eventos	100	135.6 µs	11.4 µs
Sacar los N eventos	1 000	2.29 ms	127.5 µs
Sacar los N eventos	10 000	35.74 ms	1.55 ms
Cancelar N/2 eventos	100	31.3 µs	34.1 µs
Cancelar N/2 eventos	1 000	338 µs	511 µs
Cancelar N/2 eventos	10 000	4.05 ms	8.94 ms
1 000 ciclos sacar+programar	100	2.45 ms	1.10 ms
1 000 ciclos sacar+programar	1 000	3.44 ms	1.61 ms
1 000 ciclos sacar+programar	10 000	4.80 ms	3.18 ms

Lectura. La medición no respalda que el heap sea mejor en el ciclo habitual: la lista ordenada sacó y reprogramó más rápido en los tres tamaños (con 10 000 eventos, 3.18 ms contra 4.80 ms por 1 000 ciclos). Una hipótesis, no medida, es que las inserciones y extracciones de una lista las hace C, mientras que el heap se mueve en Python puro. El heap gana al programar en bloque (2.7× con 10 000) y al cancelar (2.2× con 10 000), y su costo por ciclo crece menos: 2.0× al pasar de 100 a 10 000 eventos, contra 2.9× de la lista. Como el ciclo real tiene como mucho unos cientos de eventos pendientes (medición del ciclo del §5.2 en la sección 9), la alternativa sencilla habría bastado; el heap queda justificado solo por su garantía asintótica.

2. Caché de fichas y almacenamiento en disco

Las fichas del catálogo viven en dos niveles: una **caché LRU de capacidad fija** (una cola doble de entradas `[clave, valor, veces_en_uso]`, las más recientes al frente) y, debajo, un **archivo binario indexado en disco** (encabezado, bloques de datos, índice y pie con la posición del índice).

Problema. Cada solicitud HTTP cuenta contra el presupuesto de la cripta y además tarda del orden de 400 ms (§5). El juego necesita las mismas fichas una y otra vez: un enemigo, su arma, el objeto que suelta. La caché recibe muchas más búsquedas que inserciones, y debe cumplir tres cosas a la vez: no pasar de su capacidad, no desalojar una ficha que una sala cargada sigue usando, y no perder el trabajo hecho entre ejecuciones (el disco evita volver a gastar solicitudes).

Por qué fue buena decisión.

- **LRU:** lo que el jugador usó hace poco es lo que más probablemente vuelva a necesitar (las salas vecinas, los enemigos que lo persiguen). Desalojar lo que lleva más tiempo sin tocarse mantiene calientes las fichas de la zona actual.
- **Contador «en uso»:** una ficha que pertenece a un objeto o enemigo vivo en una sala cargada nunca se desaloja. Si todas están en uso, la nueva no se guarda (la capacidad jamás se supera) y el dato igual queda en disco. Esto evita que desalojar una ficha obligue a repetir una solicitud mientras se juega.
- **Contadores de aciertos y fallos:** salen gratis de la propia búsqueda y alimentan el reporte final del §5.1 («una caché caliente debería permitir usar menos solicitudes»).
- **Cola doble en vez de arreglo:** mover una entrada al frente al usarla no desplaza el resto del arreglo.
- **Disco con índice al final:** el archivo se escribe a un temporal y se reemplaza al terminar, así que un corte a la mitad no deja un archivo roto; el encabezado valida el formato y la versión; y una ficha o sala se lee posicionándose directo en su bloque sin leer el resto.

Complejidad. La búsqueda en la caché es lineal en su capacidad, O(c); con la capacidad por defecto (25) y la bandera `--cache-size` para ajustarla, c es pequeña y constante, y por eso la Búsqueda secuencial es aceptable aquí. En disco, localizar una ficha recorre el índice (O(k) por k claves del archivo) y luego lee un solo bloque; escribir reescribe el archivo completo, O(tamaño), lo que es aceptable porque las escrituras ocurren al traer lotes nuevos de la red, no en cada ciclo del juego.

Alternativa que también habría funcionado: tabla hash propia más lista doblemente enlazada.

	Tabla hash propia + lista doble (alternativa)	Cola doble con búsqueda lineal (elegido)
Pro	Búsqueda, inserción y desalojo O(1) en promedio, sin importar la capacidad; escala si la caché se agranda mucho	Código corto y fácil de razonar; sin función de dispersión ni colisiones; con capacidades chicas el costo es despreciable
Contra	Hay que escribir y probar la tabla (§7 prohibe `dict` en el modelo), sincronizarla con la lista y manejar colisiones y redimensionado	Cada búsqueda recorre hasta c entradas; si la capacidad creciera a miles, la operación más frecuente se volvería la más cara

Si la defensa usara una cripta mucho mayor y se pidiera una caché grande, esta es la primera pieza que habría que cambiar.

Límites conocidos.

- Con una capacidad de miles de fichas, el barrido lineal de la caché dejaría de ser despreciable.
- El archivo indexado relee el índice completo en cada consulta y reescribe todo en cada escritura; con cientos de salas guardadas, escribir lote por lote sería cada vez más caro. La salida es dividir por cripta (ya se hace) y, si hiciera falta, añadir al final en vez de reescribir.
- Los valores se guardan como JSON; es un formato sencillo y depurable a costa de tamaño en disco.

Evidencia empírica (`--bench`). Cuatro mediciones: solicitudes de red, búsqueda en la caché, lectura y escritura en disco. Primero, al recorrer un pasillo de 60 salas con 400 ms simulados por solicitud:

Estrategia de carga	Solicitudes	Espera simulada
Sala por sala, sin precarga	63	25.2 s
Actual + vecinas en cada paso (profundidad 2)	61	24.4 s
Perezosa, profundidad 2	33	13.2 s
Perezosa, lote de 10 (por defecto)	9	3.6 s

Búsqueda en la caché (µs por operación):

Fichas	Cola lineal, acierto	Cola lineal, fallo	Tabla hash propia, acierto (descartada)
25	0.74	0.44	0.17
100	2.32	1.40	0.17
1 000	24.5	14.3	0.19
5 000	132.3	76.5	0.22

Disco: recuperar una ficha liberada cuesta 0.013 ms con 25 fichas en el catálogo, 0.047 ms con 200 y 0.204 ms con 1 000, contra unos 400 ms de una solicitud real. Guardar 10 fichas nuevas (reescribe el archivo) cuesta 1.6, 1.9 y 1.4 ms con 25, 200 y 1 000 fichas. Planear la vecindad en una cripta de 1 000 salas cuesta 15 µs a un paso y 38 µs a dos.

Lectura. La carga perezosa por lotes baja las solicitudes de 63 a 9 (7×) y la espera de 25 s a 3.6 s: esa es la decisión que más pesa en el presupuesto del §5.1, mucho más que la estructura de la caché. Con la capacidad por defecto (25), buscar cuesta 0.74 µs, unas 540 000 veces menos que una solicitud, así que la tabla hash (0.17 µs) no aporta nada medible; solo empieza a importar con cientos de fichas (24.5 µs contra 0.19 µs con 1 000, 600× con 5 000), que es el umbral para cambiarla. Leer del disco es unas 2 000 veces más barato que repetir la solicitud. El supuesto de que reescribir el archivo completo se encarece con el tamaño no se vio hasta 1 000 fichas (1.4 a 1.9 ms).

Caché en una partida completa. Se simuló un recorrido de 150 pasos por una rejilla de 10×10 salas (serpiente completa y vuelta atrás por la primera mitad), con 40 tipos de enemigo y 30 de objeto, precargando vecinas, reteniendo las fichas de los enemigos de la sala actual y liberando al salir:

Capacidad	Aciertos	Fallos	Tasa de aciertos	Solicitudes de red
5	82	938	8.0 %	19
10	316	704	31.0 %	19
25 (por defecto)	487	533	47.7 %	19
100	950	70	93.1 %	19
1 000	950	70	93.1 %	19

La tasa de aciertos sube con la capacidad hasta cubrir las unas 70 fichas distintas de la cripta (100 ya es el máximo), pero las solicitudes de red son las mismas con cualquier capacidad: lo que la caché no retiene lo absorbe el disco, que cuesta unos 0.2 ms por ficha. Es decir, la capacidad de la caché decide cuántas lecturas de disco se evitan, no cuántas solicitudes de red se gastan; el presupuesto del §5.1 lo protege el disco y la carga por lotes.

3. Historial de retroceso (pergamino)

El historial es una **lista doblemente enlazada de nodos de cambio que se usa como pila**: cada nodo guarda uno o más cambios (el valor anterior y el nuevo de lo que se modificó), y un nodo especial marca dónde empieza cada acción del jugador. Solo se conservan las últimas 5 acciones.

Problema. El pergamino de retroceso debe devolver el juego al instante anterior a la última acción del jugador, y eso incluye lo que pasó por consecuencia: golpes de enemigos, una trampa que se activó, una puerta que se programa para cerrarse, el reloj y hasta eventos encolados o desencolados en la agenda. Cada ataque, recolección, movimiento, cambio de velocidad o evento de puerta deja una huella distinta, y la operación que se repite es **agregar al final** (cada vez que algo cambia) y **quitar del final** (al deshacer), mientras que descartar la acción más antigua ocurre solo cuando se pasa del límite.

Por qué fue buena decisión.

- **Guardar diferencias en vez de copias:** cada cambio se registra como «atributo X pasó de a a b». Registrar un golpe cuesta unos pocos campos, no una copia del mundo; el costo de guardar crece con lo que realmente cambió, no con el tamaño de la cripta.
- **Pila sobre lista doble:** agregar al final y sacar del final son O(1), sin desplazar nada, y la lista no tiene un tamaño máximo que reservar de antemano ni que redimensionar.
- **Marcadores de acción:** permiten deshacer una acción completa (la del jugador y todo lo que provocó hasta la siguiente decisión) retirando nodos hasta encontrar el marcador, sin buscar ni contar.
- **Límite de 5 acciones:** acota la memoria sin importar cuánto dure la partida. Descartar la más antigua solo recorre los nodos de esa acción, no todo el historial.
- **Acción rechazada:** como el marcador se abre antes de ejecutar, una acción que el juego rechaza se descarta sin gastar un lugar del historial ni mover el reloj (§2.13), y el recorte se difiere hasta que la acción se acepta para no perder una acción válida anterior.

Complejidad.

Operación	Costo
Registrar un cambio	O(1)
Deshacer la última acción	O(k), con k = cambios de esa acción
Descartar la acción más antigua	$O(k_a)$, con k_a = cambios de esa acción
Memoria total	Acotada por 5 acciones, sin importar la duración de la partida

Alternativa que también habría funcionado: política de instantáneas (copia del estado antes de cada acción) en un buffer circular de 5.

	Instantáneas en buffer circular (alternativa)	Diferencias en lista doble (elegido)
Pro	Deshacer es copiar de vuelta: no hay que escribir código de reversión por cada tipo de acción, así que olvidar un caso es imposible	La memoria y el tiempo de registro dependen de lo que cambia, no del tamaño del mundo
Contra	Cada acción copia todo el estado (salas cargadas, inventario, agenda): costo proporcional al mundo en cada paso, y copiar la agenda de eventos conserva objetos duplicados	Cada tipo de cambio necesita su propio código de reversión; si se agrega un efecto nuevo y se olvida registrarlo, el pergamino lo deja sin deshacer

Para un mundo pequeño las instantáneas habrían sido más seguras; las diferencias ganan cuando la cripta tiene muchas salas cargadas, que es el caso de la defensa.

Límites conocidos.

- El mayor riesgo no es el rendimiento sino la **completitud**: cada mecánica nueva (el veneno y la regeneración del §2.10, ya implementados, registran sus cambios y sus eventos de la agenda, y están cubiertos por pruebas de retroceso) debe registrar sus cambios o el retroceso quedará inconsistente. Por eso las pruebas de retroceso comparan una foto completa del estado antes y después.
- El historial es un único objeto compartido por partida (patrón singleton); no admite dos partidas simultáneas en el mismo proceso.

- El historial no se guarda en disco; una partida cargada empieza con el historial vacío.

Evidencia empírica (`--bench`). En ambas estrategias, deshacer una acción devuelve el mundo a su estado anterior (verificado). Registrar 200 acciones, según el tamaño del mundo y los cambios por acción:

Entidades en el mundo	Cambios por acción	Diferencias (elegido)	Instantáneas (alternativa)
30	1	302 µs	234 µs
30	10	1.52 ms	256 µs
300	1	286 µs	1.30 ms
300	10	1.53 ms	1.40 ms
3 000	1	292 µs	12.88 ms
3 000	10	1.56 ms	12.85 ms

Deshacer una acción en un mundo de 300 entidades: con 1 cambio, 1.5 µs contra 11.4 µs; con 10 cambios, 4.5 µs contra 11.5 µs; con 100 cambios, 34.1 µs contra 11.7 µs (diferencias contra instantáneas).

Lectura. El costo de las diferencias no depende del mundo (unos 290 µs o 1.5 ms por 200 acciones en los tres tamaños), mientras que el de las instantáneas crece 55× de 30 a 3 000 entidades. Con un mundo de 300 entidades o más, que es el caso de la cripta cargada, las diferencias empatan o ganan al registrar (hasta 44× con 3 000) y deshacen de 2.6 a 7.6× más rápido las acciones típicas de 1 a 10 cambios. Las instantáneas ganan en mundos de 30 entidades (6× con 10 cambios por acción) y al deshacer acciones de 100 cambios, algo raro en el juego. Con la memoria acotada a 5 acciones, lo que se mide es solo tiempo: ninguna de las dos crece con la duración de la partida.

4. Inventario del jugador

El inventario es una **lista doblemente enlazada con un cursor** (`actual`) que apunta al objeto seleccionado, con capacidad máxima fija. Los dos primeros nodos son los objetos equipados (arma y armadura).

Problema. El jugador recorre su mochila con «siguiente» y «anterior», usa o suelta el objeto bajo el cursor, recoge objetos al final y equipa o desequipa moviendo un objeto a los primeros lugares. El pergamino de retroceso, además, debe poder devolver un objeto a la **posición exacta** donde estaba. Las operaciones que se repiten son mover el cursor, agregar al final y quitar el objeto seleccionado; buscar un objeto por identidad y reinsertar en una posición dada ocurren solo al usar, equipar o deshacer.

Por qué fue buena decisión.

- Mover el cursor al vecino, agregar al final y desenlazar el nodo seleccionado son O(1) y no desplazan al resto, que es el patrón de uso real de una mochila.
- Quitar un objeto en medio no deja huecos que compactar ni obliga a renumerar índices.
- La capacidad se controla con un contador, sin reservar ni redimensionar memoria; el límite de peso o de espacio es una simple comparación.
- Los objetos equipados son simplemente los dos primeros nodos, de modo que «equipar» es reenlazar un nodo y «deshacer» es reenlazarlo en su índice anterior, sin una estructura extra que mantener sincronizada.
- Las vistas ordenadas (por peso, valor o nombre) no tocan la lista: copian los objetos a una lista de trabajo y la ordenan ahí, así el orden «real» de la mochila, que importa para los equipados y para el retroceso, nunca cambia.

Complejidad.

Operación	Costo	Frecuencia
Mover el cursor	O(1)	Muy frecuente
Agregar al final	O(1)	Frecuente
Soltar o usar el objeto del cursor	O(1)	Frecuente
Buscar un objeto por identidad	O(n)	Al usar, equipar o deshacer
Reinsertar en un índice (retroceso)	O(n)	Solo al usar el pergamino
Vista ordenada	O(n) para copiar + costo del ordenamiento	Al consultar la mochila

n es la capacidad del inventario, que es chica y fija; por eso los recorridos lineales son aceptables.

Alternativa que también habría funcionado: arreglo contiguo con índice de cursor.

	Arreglo con índice (alternativa)	Lista doble con cursor (elegido)
Pro	Acceso por índice O(1) y buena localidad en memoria; reinsertar en una posición para el retroceso es un solo código; las vistas ordenadas se trabajan directo sobre el arreglo	Quitar y reenlazar un nodo en medio no desplaza elementos; el cursor siempre apunta a un nodo válido sin recalcular índices
Contra	Quitar en medio desplaza todos los elementos siguientes, O(n); hay que ajustar el índice del cursor tras cada quita; el espacio se reserva por adelantado	Sin acceso por posición: ir al índice i recorre i nodos; cada nodo agrega dos referencias de memoria

Con una capacidad de unas pocas decenas de objetos, ambas cuestan microsegundos; medido, el arreglo es más rápido en casi todo, así que la lista se mantiene por claridad (el cursor y los equipados se expresan sin índices) y no por velocidad.

Límites conocidos. Si la capacidad del inventario creciera a cientos o miles de objetos, las búsquedas lineales por identidad y las reinserciones del retroceso dejarían de ser despreciables y habría que añadir un índice auxiliar. El ordenamiento ya vive en la capa de lógica (Logica/ordenamiento.py), así que el inventario no depende del controlador, como pide la separación del MVC del §6.

Evidencia empírica (`--bench`). Quitar y reinsertar en su índice deja el mismo orden en ambas estructuras (verificado). Con la búsqueda por identidad que hace el juego al usar, equipar o deshacer:

Capacidad	Operación	Lista doble (elegido)	Arreglo (alternativa)
10	Agregar N objetos	2.3 µs	1.0 µs
10	Mover el cursor de punta a punta	0.8 µs	0.9 µs
10	200× quitar y reinsertar el del medio	191 µs	79 µs

Capacidad	Operación	Lista doble (elegido)	Arreglo (alternativa)
100	Agregar N objetos	20.8 µs	7.2 µs
100	Mover el cursor de punta a punta	7.0 µs	8.2 µs
100	200× quitar y reinsertar el del medio	552 µs	308 µs
1 000	Agregar N objetos	193.5 µs	59.7 µs
1 000	Mover el cursor de punta a punta	57.7 µs	70.5 µs
1 000	200× quitar y reinsertar el del medio	4.72 ms	2.84 ms

Lectura. La medición contradice la expectativa: el arreglo es de 2.3 a 3.2× más rápido al agregar y de 1.7 a 2.4× al quitar y reinsertar, y la lista solo gana al mover el cursor (el arreglo tarda de 12 a 22 % más). La ventaja teórica de no desplazar elementos no aparece porque la búsqueda por identidad es O(n) en las dos y domina. Aun con capacidad 1 000, 200 quitas y reinserciones cuestan 4.7 ms (24 µs cada una), así que ninguna es un cuello de botella.

5. Ordenamiento

Se implementaron **dos algoritmos propios y estables, ordenamiento por inserción y ordenamiento por mezcla (merge sort)**, y una función que elige entre ellos: hasta 15 elementos usa inserción; de 16 a 64 prueba la inserción con un presupuesto de 4·n desplazamientos y, si los datos no estaban casi ordenados, se rinde y usa mezcla; con más de 64 usa mezcla directamente. Ambos reciben una función de clave, así que sirven para cualquier criterio sin conocer el tipo de dato.

Problema. El §4.9 pide ordenar en contextos distintos: las vistas del inventario (por peso, valor o nombre, sobre listas de pocas decenas de objetos), la tabla de puntajes (una lista que crece partida tras partida, con clave compuesta de varios campos) y listas auxiliares. Los tamaños, el orden previo de los datos y el costo de comparar cambian de un caso a otro, y por eso el enunciado exige dos algoritmos con características distintas y una elección automática. Prohibe `list.sort()` y `sorted()`.

Por qué fue buena decisión.

- **Inserción:** en listas pequeñas o casi ordenadas hace muy pocas comparaciones y mueve elementos sin memoria extra. Es el caso de una mochila que ya estaba ordenada y recibió un objeto nuevo.
- **Mezcla:** garantiza O(n log n) sin importar cómo lleguen los datos, que es lo que se necesita para una tabla de puntajes que crece. Es estable, así que ordenar por varios criterios sucesivos conserva el orden anterior entre iguales.
- **Estabilidad en ambos:** el resultado es determinista para empates, y eso importa para que dos corridas con los mismos datos muestren la misma tabla.
- **La función de clave:** extrae el dato a comparar una sola vez por criterio de uso y permite que la lista enlazada del inventario no sepa nada del ordenamiento, que recibe una copia en lista.

Complejidad.

Algoritmo	Mejor caso	Peor caso	Memoria extra	Cuándo conviene
Inserción	O(n), datos ya ordenados	$O(n^2)$	O(1)	Pocos elementos o casi ordenados
Mezcla	O(n log n)	O(n log n)	O(n)	Muchos elementos, orden desconocido

El umbral de 15 elementos es el punto donde el costo cuadrático de la inserción todavía es menor que el de crear las sublistas de la mezcla. **Medido con `--bench`, con datos aleatorios el cruce cae entre 15 y 50 elementos, así que el 15 resulta razonable;** los casos donde la regla por tamaño falla se detallan en la evidencia de abajo.

Alternativa que también habría funcionado: un solo algoritmo, ordenamiento rápido (quicksort) con pivote mediana de tres, o montículo (heapsort).

	Un algoritmo único (alternativa)	Inserción + mezcla según tamaño y orden previo (elegido)
Pro	Menos código y una sola estrategia que probar y explicar; quicksort ordena en el lugar y es muy rápido en promedio	Cada algoritmo trabaja en el rango donde es bueno, y el requisito del §4.9 de dos algoritmos con características distintas se cumple de forma natural

	Un algoritmo único (alternativa)	Inserción + mezcla según tamaño y orden previo (elegido)
Contra	Quicksort no es estable y puede degradarse a $O(n^2)$ con ciertos datos; heapsort es estable solo con trabajo extra y tiene mala localidad; ninguno aprovecha que los datos ya vengan ordenados	Hay dos caminos que probar; la elección mira el tamaño y el orden previo, pero no el costo de comparar

Límites conocidos.

- **Cómo leimos el §4.9.** El enunciado pide que el programa escoja el algoritmo «de acuerdo con propiedades medibles de los datos» y que la decisión se sustente con mediciones propias que «consideren tamaños pequeños, datos casi ordenados y comparaciones costosas». Entendimos que lo primero exige una regla que mire propiedades de los datos (la nuestra mira el tamaño y el orden previo) y que lo segundo exige medir esos tres casos, no que la regla decida por cada uno. Por eso las comparaciones costosas están medidas (clave compuesta de cuatro campos, en todas las tablas de esta sección) pero no entran en la regla. Esa lectura se cumple con certeza; la más estricta, que la regla también decida por el costo de comparar, quedaría cubierta calculando la clave una sola vez por elemento, y no se implementó porque medir el costo en tiempo de ejecución haría la elección distinta entre corridas y chocaría con el determinismo del §4.1. El orden previo solo se prueba hasta 64 elementos, porque con más la mezcla ya gana incluso con datos casi ordenados; con listas grandes casi ordenadas en un grado mucho mayor al medido (por ejemplo, un solo elemento fuera de lugar) la inserción seguiría siendo mejor y la regla no la usa.
- La mezcla copia sublistas en cada nivel; con listas de decenas de miles de elementos, esa memoria extra y las copias podrían notarse frente a una mezcla que use un solo arreglo auxiliar.
- El ordenamiento hoy se usa en el inventario y en la tabla de puntajes; los índices del archivo de guardado y las listas auxiliares del catálogo (§4.9) todavía no pasan por él.

Evidencia empírica (`--bench`). Los cuatro algoritmos dan el mismo resultado con clave compuesta (verificado con 300 datos). Tiempos para una selección de tamaños, entradas y costos de comparación (la inserción se omite con N > 2 000 aleatorio o inverso por su costo $O(n^2)$); «Automática» es la regla que tenía el juego al medir (hasta 15 inserción, si no mezcla):

Clave	N	Entrada	Inserción	Mezcla	Automática	Quicksort (alternativa)
Barata	15	Aleatoria	5.5 µs	7.6 µs	5.4 µs	4.5 µs
Barata	15	Inversa	8.4 µs	6.5 µs	8.4 µs	4.5 µs
Barata	50	Aleatoria	48.5 µs	33.5 µs	33.5 µs	18.9 µs
Barata	50	Casi ordenada	11.8 µs	28.3 µs	28.5 µs	19.4 µs
Barata	50	Inversa	88.4 µs	26.6 µs	26.6 µs	20.9 µs
Barata	500	Aleatoria	4.71 ms	487 µs	486 µs	284 µs
Barata	500	Inversa	8.83 ms	344 µs	335 µs	379 µs
Barata	5 000	Aleatoria	omitida	6.71 ms	6.69 ms	3.93 ms
Barata	5 000	Casi ordenada	53.6 ms	5.89 ms	5.93 ms	4.94 ms
Barata	5 000	Inversa	omitida	4.35 ms	4.37 ms	5.83 ms
Cara (4 campos)	15	Aleatoria	11.4 µs	10.5 µs	11.5 µs	7.3 µs
Cara (4 campos)	50	Aleatoria	78.5 µs	50.6 µs	49.7 µs	31.8 µs
Cara (4 campos)	500	Aleatoria	7.83 ms	742 µs	741 µs	476 µs
Cara (4 campos)	5 000	Aleatoria	omitida	10.26 ms	10.24 ms	6.43 ms
Cara (4 campos)	5 000	Casi ordenada	804 ms	10.11 ms	10.08 ms	6.56 ms

Lectura. Tres conclusiones. Primera, el umbral de 15 se confirma para datos aleatorios: a N = 15 la inserción gana (5.5 contra 7.6 µs) y a N = 50 pierde (48.5 contra 33.5 µs); con clave cara hay empate en 15. Segunda, la regla que solo mira el tamaño falla con datos casi ordenados: a N = 50 elige la mezcla (28.5 µs) cuando la inserción tardaba 11.8 µs (2.4× menos), y con clave barata a N = 15 inversa elige la inserción (8.4 µs) cuando la mezcla tardaba 6.5 µs; eso respalda decidir por propiedades medibles de los datos, como pide el §4.9. Tercera, la alternativa sencilla de un solo algoritmo fue la más rápida en 15 de las 18 combinaciones con N ≥ 50, entre 1.2 y 1.8× sobre la mezcla, pero la mezcla gana con entrada inversa (344 contra 379 µs con N = 500; 4.35 contra 5.83 ms con N = 5 000). Con las listas reales del juego (decenas de elementos) la diferencia es de microsegundos, así que el quicksort habría alcanzado en tiempo; la mezcla se mantiene por ser estable (importa para los empates de la tabla de puntajes) y por su peor caso garantizado, no por ser más rápida.

Regla actual del juego, medida. Para cumplir el §4.9 la elección dejó de mirar solo el tamaño: entre 16 y 64 elementos prueba la inserción con un presupuesto de 4·n desplazamientos y se rinde a la mezcla si lo agota. Comparada con la regla anterior (solo tamaño), clave barata salvo donde se indica:

Caso	Regla anterior	Regla actual
N = 50, casi ordenada	27.7 µs	11.2 µs
N = 50, aleatoria	33.1 µs	47.5 µs
N = 50, inversa	26.6 µs	41.0 µs
N = 50, casi ordenada, clave cara	47.7 µs	87.2 µs
N = 500, casi ordenada	401.6 µs	394.6 µs
N = 5 000, casi ordenada	5.84 ms	5.84 ms

Lectura. La regla nueva gana 2.5× donde el enunciado lo pide (datos casi ordenados y pocos elementos) y pierde entre 14 y 40 µs cuando la inserción gasta su presupuesto antes de rendirse. La versión sin tope de tamaño perdía de 18 a 36 % con 500 y 5 000 elementos, por eso la prueba se limita a 64. Con presupuestos de 2·n o 1·n desaparece la ganancia con 50 casi ordenados, así que se dejó 4·n. El caso de clave cara es peor porque la clave del bench revuelve el orden y los datos no quedan casi ordenados; el costo de comparar sigue sin entrar en la elección.

Tabla de puntajes (clave compuesta, 1 000 y 10 000 registros). El archivo crece por anexado, así que en la práctica la tabla llega casi ordenada con partidas nuevas al final.

N	Entrada	Inserción	Mezcla	Regla actual	Quicksort (alternativa)
1 000	Aleatoria	omitida	1.72 ms	1.69 ms	1.10 ms
1 000	Casi ordenada	2.97 ms	1.09 ms	1.09 ms	2.61 ms
10 000	Aleatoria	omitida	23.21 ms	23.18 ms	14.87 ms
10 000	Casi ordenada	omitida	14.14 ms	14.19 ms	19.87 ms

Quicksort es 1.6× más rápido con datos aleatorios, pero con la tabla casi ordenada tarda de 1.4 a 2.4× más que la mezcla, y como no es estable dejó 8 de 2 000 registros en distinto lugar que la mezcla por empates en la clave. Para esta tabla la mezcla es la mejor opción por estabilidad y por el patrón real de los datos.

6. Guardado binario de la partida

La partida se guarda en **un archivo binario de formato propio**: encabezado con marca y versión, bloques de datos (uno por sección del estado y uno por sala), un índice de secciones, una **tabla de salas ordenada por número** (posición, largo y CRC de cada bloque) y un pie de tamaño fijo que apunta a todo lo anterior. Los valores se codifican con una marca de un byte por tipo y enteros de largo variable (7 bits por byte, con signo en zigzag), sin `pickle`, sin `json` y sin diccionarios.

Problema. El §4.10 pide tres cosas a la vez: formato propio, **recuperar una sala específica sin leer el archivo completo** y detectar un archivo de otra versión del programa; y además comparar tamaño y tiempo de carga contra guardar lo mismo en JSON. Una partida guarda las salas cargadas con sus enemigos, trampas, salidas y objetos, el inventario, el reloj con su agenda, la bitácora y el contador de acciones; lo que cambia de una partida a otra es sobre todo la cantidad de salas.

Por qué fue buena decisión.

- **Pie fijo al final:** el lector salta directo al final, lo valida (marca de cierre y coherencia de posiciones) y desde ahí llega al índice sin recorrer nada. Un archivo cortado a la mitad se detecta de inmediato porque le falta la marca.
- **Tabla de salas ordenada con entradas de tamaño fijo:** como cada entrada ocupa lo mismo, se puede saltar a la i-ésima sin leer las anteriores, y como está ordenada se encuentra una sala con **búsqueda binaria en O(log s)** lecturas pequeñas, para luego leer solo ese bloque.
- **CRC por bloque:** una sala dañada se detecta al leerla, y no contamina el resto de la partida.
- **Enteros de largo variable:** los números chicos (casi todos: vida, ids, tiempos) ocupan un byte, lo que reduce el tamaño frente a un ancho fijo de ocho bytes o a texto.
- **Escritura a un temporal y reemplazo:** si el programa se cierra a mitad de guardar, el archivo anterior sigue intacto.
- **La versión del formato en el encabezado** hace que un archivo de otra versión se rechace con un mensaje claro en vez de leerse mal (por ejemplo, al sumar el contador de acciones del jugador se subió la versión del formato).
- **El modelo no conoce el archivo:** el estado se exporta como datos simples y es un adaptador el que los escribe, así que cambiar el formato no toca la lógica del juego.

Complejidad.

Operación	Costo
Guardar la partida	O(tamaño del estado), una sola pasada
Cargar una sala	O(log s) lecturas en la tabla + una lectura del bloque, con s salas guardadas
Cargar la partida completa	O(tamaño del estado)
Listar las salas guardadas	O(s), sin leer ningún bloque de sala

Alternativa que también habría funcionado: registros de tamaño fijo con struct, una fila por entidad.

	Registros fijos con `struct` (alternativa)	Codificación con marcas y enteros variables (elegido)
Pro	Leer cualquier entidad es calcular una posición: acceso O(1) sin tabla; formato muy simple de documentar y verificar	Compacto; los campos opcionales y las listas de largo variable no desperdician espacio; un tipo nuevo se agrega al final sin reescribir el resto
Contra	Los textos y listas de largo variable (nombres, inventario, mensajes) obligan a reservar un ancho máximo y desperdiciar espacio, o a mantener tablas auxiliares; agregar un campo cambia el tamaño de todos los registros	Para llegar a una sala hace falta la tabla; no se puede calcular su posición directamente porque los bloques miden distinto

La tabla de puntajes sí usa registros fijos (nombre de hasta 15 caracteres, texto de largo fijo para cripta y versión), porque allí todos los registros son del mismo tipo y agregar uno nuevo es escribir al final.

Límites conocidos.

- Guardar siempre escribe todo el estado; con cientos de salas cargadas, un guardado incremental (solo las salas que cambiaron) sería más rápido pero más complejo.
- Las secciones del estado se buscan en orden porque son pocas; si creciera su número, convendría indexarlas igual que las salas.
- El historial del pergamino **no** se guarda: una partida cargada empieza sin poder retroceder. Es una limitación aceptada y documentada.

- No hay compresión; para partidas muy grandes se podría añadir por bloque.

Evidencia empírica (`--bench`). Cargar el binario devuelve exactamente el estado guardado y un archivo de otra versión se rechaza (ambos verificados). Comparación contra guardar lo mismo en JSON (binario / JSON):

Salas	Tamaño en disco	Guardar la partida	Cargar todo	Conseguir una sala
10	8 107 B / 18 852 B (2.33×)	520 µs / 736 µs	1.13 ms / 69 µs	30 µs / 67 µs
100	42 296 B / 120 030 B (2.84×)	2.54 ms / 4.17 ms	5.18 ms / 427 µs	30 µs / 446 µs
1 000	395 092 B / 1 147 831 B (2.91×)	23.3 ms / 40.3 ms	46.9 ms / 4.28 ms	38 µs / 4.28 ms

En «cargar todo» el binario construye los objetos de datos y el JSON solo analiza el texto, así que la comparación favorece al JSON; para una sala, el JSON tiene que leer todo el archivo.

Lectura. El binario gana en lo que el §4.10 pide demostrar: pesa de 2.3 a 2.9× menos, guarda de 1.4 a 1.7× más rápido y trae una sola sala 2.2× más rápido con 10 salas, 15× con 100 y 113× con 1 000, porque su tiempo casi no crece con el número de salas (30 a 38 µs) mientras que el del JSON crece en línea. **Pero la medición contradice que cargar sea más rápido:** cargar la partida completa en binario es de 11 a 16× más lento que `json.loads`, que está escrito en C, mientras que el decodificador propio corre en Python. Con 1 000 salas son 47 ms contra 4 ms; el costo solo se paga al reanudar una partida, no en cada ciclo, y no cambia el veredicto del formato, pero hay que decirlo así en la defensa.

7. Bitácora de partida y reproducción (`--replay`)

Cada acción del jugador se guarda en dos lugares con roles distintos: en memoria, una **lista doblemente enlazada de acciones en orden cronológico** (`LogEventos`); y en disco, un **archivo de texto de solo anexado** (`AlmacenLogTexto`) con un encabezado de cuatro líneas (cripta, semilla, versión de cripta, versión de catálogo) y una línea por acción.

Problema. El §4.11 exige que cada acción quede registrada en un archivo de texto de solo anexado y que `--replay partida.log` reproduzca la partida completa sin interacción. Para que la reproducción sea fiel, el juego tiene que ser determinista (mismas acciones,

misma semilla y mismos datos dan la misma partida) y, si la cripta o el catálogo cambiaron desde que se grabó, el programa debe reportar que no puede garantizar una reproducción fiel en lugar de reproducir en silencio contra datos distintos.

Por qué fue buena decisión.

- **Agregar al final en O(1) y recorrer en orden:** una acción nueva es enlazar un nodo, sin copiar ni desplazar lo anterior. El mismo orden sirve para escribir en disco y para reproducir.
- **Texto de solo anexado:** cada acción se escribe en el instante en que ocurre y el archivo se abre en modo exclusivo, así que ni un cierre inesperado ni un nombre repetido pueden perder o pisar un log. Es legible a simple vista y se puede revisar con cualquier editor.
- **Una línea por acción con los datos mínimos:** tipo de acción, tiempo, actor y referencias por identificador (nunca objetos completos). Una referencia ligera se resuelve contra el estado de la partida que se reproduce, así que el log no necesita crecer con el mundo.
- **Encabezado con versiones:** permite comprobar antes de empezar que la cripta y el catálogo siguen siendo los mismos que cuando se grabó.
- **Un solo generador de azar por partida:** la semilla del encabezado reconstruye la misma secuencia de tiradas, de modo que el orden en que se resuelven los eventos (garantizado por el desempate de la agenda) fija el resultado, y no el orden en que lleguen respuestas de red.
- **El modelo no escribe archivos:** avisa cada acción a un registrador, y es el adaptador el que la guarda. Así se pudo separar la E/S de la lógica y reproducir sin crear un log nuevo.

Complejidad.

Operación	Costo
Registrar una acción (memoria)	O(1)
Anexar una línea al archivo	O(1), una escritura pequeña
Reproducir la partida	O(a), con a acciones, más el costo de cada ciclo de eventos
Leer y validar el encabezado	O(1)

Para interpretar cada línea (que es JSON) se usa el diccionario que produce `json.loads` solo de forma transitoria, para leer el texto; no se guarda ni se usa como índice, así que respeta el §7.

Alternativa que también habría funcionado: arreglo dinámico (`list`) en memoria y un formato binario para el log.

	Arreglo dinámico + log binario (alternativa)	Lista enlazada + log de texto (elegido)
Pro	El arreglo da acceso por índice O(1) y mejor localidad; un log binario es más compacto y más rápido de leer	Anexar nunca copia ni redimensiona; el log de texto se puede leer, revisar y editar a mano para armar escenarios de prueba, que es justo el uso de `--replay` en la revisión
Contra	Un arreglo que crece copia sus elementos cada cierto tiempo; un binario no se puede inspeccionar sin herramienta y es más difícil de depurar	Cada nodo agrega referencias de memoria; el texto ocupa más que el binario y cada línea hay que interpretarla al reproducir

Límites conocidos.

- Una partida de cientos de miles de acciones haría que la lista en memoria y el texto pesen más de lo óptimo; para el tamaño realista de una partida no es un problema.
- La reproducción es fiel solo mientras las versiones coincidan; con una versión distinta el programa se niega a reproducir, a propósito. No guardamos copias locales de versiones anteriores, así que no hay camino de recuperación más allá de reportarlo.
- Una acción rechazada por el juego no se registra (no consume tiempo ni modifica nada), así que el log contiene solo lo que efectivamente ocurrió.

Evidencia empírica (`--bench`). Ambas bitácoras de mensajes dejan los mismos 20 últimos, en el mismo orden (verificado). Estructura elegida / alternativa:

Medición	Elegido	Alternativa
20 000 mensajes con la bitácora llena	Cola enlazada: 4.70 ms	Buffer circular: 1.48 ms
1 000 acciones registradas en memoria	Lista enlazada: 214 µs	Arreglo: 21.3 µs

Medición	Elegido	Alternativa
10 000 acciones registradas en memoria	Lista enlazada: 2.62 ms	Arreglo: 209 µs
300 líneas anexadas al `.log`	Abriendo y cerrando en cada acción: 944 µs	Archivo abierto una sola vez: 8.6 µs
Reproducir un log de 300 acciones (incluye armar el juego)	49.7 ms	no medido

Lectura. La medición contradice la expectativa de que anexar a una lista enlazada fuera comparable a un arreglo: en memoria el arreglo es de 3 a 12× más rápido (la cola de mensajes, 3.2×; las acciones, 10 a 12.5×). En valores absolutos son 0.2 µs contra 0.02 µs por acción, irrelevantes frente a un ciclo de milisegundos, por lo que la alternativa sencilla habría bastado. Abrir y cerrar el `.log` en cada acción cuesta 110× más (3.1 µs por línea), y es el precio de la durabilidad: cada acción queda escrita aunque el programa se cierre de golpe. Reproducir una partida de 300 acciones toma unos 50 ms, así que la reproducción no domina el costo.

8. El mundo: salas, vecindad, mensajes recientes y puntajes

Cuatro piezas más pequeñas completan el juego, y se agrupan por el tipo de estructura que comparten: un **arreglo indexado por número de sala**, una **cola** (usada tanto para recorrer vecinos como para la bitácora de mensajes) y una **tabla de registros fijos** para los puntajes.

Salas como arreglo indexado por número. La lista de salas del juego tiene un puesto por cada número de sala, y las salas que todavía no se cargaron quedan vacías. Encontrar la sala del jugador, la de un enemigo o la de una trampa es un acceso directo O(1), y es la consulta más frecuente del ciclo (cada acción y cada evento de enemigo la usa). Esto cumple el §7, porque se usa `list` como almacenamiento contiguo de bajo nivel y no como índice de un diccionario. **Alternativa:** una lista enlazada de salas cargadas. **Pro:** no reserva puestos para salas que nunca se visitan, útil si la cripta fuera enorme y casi toda quedara sin cargar. **Contra:** encontrar una sala pasa a ser O(s) y habría que recorrerla en cada evento. **Límite:** el arreglo asume que los números de sala son consecutivos desde 1; una cripta desconocida de la defensa con ids muy dispersos desperdiciaría espacio o fallaría, y habría que pasar a una estructura ordenada con búsqueda binaria.

Cola para recorrer vecinos y para los últimos mensajes. Las salas vecinas se encuentran con un recorrido por niveles sobre una cola de doble extremo, que visita primero todas las salas a un paso, luego a dos, y así hasta la profundidad pedida; el orden por distancia es lo que decide qué salas entran primero en un lote de contenido, que es una sola solicitud con hasta 10 salas. La bitácora de mensajes que ve el jugador es una **cola enlazada de 20 mensajes**: agregar uno nuevo es enlazarlo al final y, si se pasa de 20, soltar el del frente, ambas en O(1), sin desplazar los 19 anteriores (el requisito literal del §4.8). **Alternativa:** un buffer circular sobre un arreglo de tamaño fijo. **Pro:** memoria reservada una sola vez y mejor localidad; no crea un nodo por mensaje. **Contra:** quitar un mensaje del medio (lo hace el retroceso al deshacer una acción) obliga a desplazar elementos, mientras que en la cola enlazada solo se reenlazan dos nodos. **Límite:** la comprobación de si una sala ya fue alcanzada recorre la lista de alcanzadas, O(n) por sala; con criptas de miles de salas conviene reemplazarla por una marca por número de sala.

Tabla de puntajes en registros fijos. Cada partida terminada agrega un registro de tamaño fijo al final del archivo (nombre de hasta 15 caracteres, cripta, versión, acciones, enemigos derrotados, tiempo final y si fue victoria). Agregar es O(1) y no reescribe el archivo, y leer todos es una pasada. La tabla se ordena en memoria con los algoritmos de la sección 5 y una clave compuesta: primero las victorias, luego menos tiempo, menos acciones y más enemigos derrotados. **Alternativa:** guardar los registros como texto (CSV o JSON). **Pro:** legible y editable a mano. **Contra:** hay que interpretar cada línea y un campo mal escrito rompe la lectura; los registros fijos se validan solos por tamaño. **Límite:** con muchos miles de partidas, ordenar en cada consulta empezaría a costar y convendría mantener el archivo ordenado o leer solo los mejores N.

Evidencia empírica (`--bench`). Con 200 consultas por número a salas ya cargadas, el arreglo indexado tarda 1.6 µs con 10, 100 y 1 000 salas; la lista enlazada, 26 µs, 162 µs y 1.56 ms (unas 1 000× más con 1 000 salas). El recorrido por niveles a profundidad 3 cuesta 3.4 µs con 10 salas, 10.9 µs con 100 y 75 µs con 1 000 (15 salas alcanzadas): crece con el tamaño de la cripta porque cada sala se busca linealmente en el esqueleto, pero sigue siendo despreciable frente a los 400 ms de una solicitud. La cola de mensajes contra un buffer circular está en la sección 7 (el buffer circular es 3.2× más rápido) y la tabla de puntajes en la sección 5.

9. Qué se degrada primero si la cripta crece 50 veces

Las decisiones anteriores están pensadas para la cripta de referencia; si la de la defensa tiene 50 veces más salas y actores, lo primero en notarse no es la agenda de eventos (el heap escala bien) sino las búsquedas lineales sobre datos que crecen con la cripta. En

orden de aparición esperado:

1. **Búsqueda de una sala en el esqueleto de la cripta.** Se hace recorriendo la lista de salas en cada consulta, y el recorrido por vecindad la repite para cada sala visitada. Con miles de salas la búsqueda crece, pero medido con 1 000 salas planear la vecindad cuesta solo 15 µs a un paso y 38 µs a dos, despreciable frente a los 400 ms de una solicitud; se notaría con muchas más. Solución: construir una vez un arreglo indexado por número de sala.
2. **Archivo indexado de fichas y criptas.** Cada lectura relee el índice y cada escritura reescribe el archivo entero. Solución: guardar el índice en memoria durante la sesión y añadir al final en lugar de reescribir.
3. **Caché con búsqueda lineal.** Aceptable con la capacidad actual; si la cripta grande exigiera una caché de cientos de fichas, es la operación con más búsquedas que inserciones y la primera candidata a pasar a una tabla hash propia.
4. **Comprobación de salas ya alcanzadas** en el recorrido por vecindad (O(n) por sala). Solución: una marca por número de sala.
5. **Reajuste de velocidad** (barrido por actor en la agenda): solo afecta cuando hay muchos actores y muchos cambios de velocidad seguidos; un índice por actor lo resolvería.
6. **Guardado completo:** crece con las salas cargadas, no con las existentes, así que se mantiene acotado por la precarga y por el tamaño de la caché.

Lo que **no** se degrada con el tamaño de la cripta: agregar y sacar eventos de la agenda (O(log n)), registrar y deshacer cambios (O(1) y acotado a 5 acciones), anexar a la bitácora y al log, el acceso a una sala ya cargada por número y la lectura de una sala del guardado (búsqueda binaria en la tabla).

Evidencia del ciclo de eventos (§5.2, `--bench`). El límite es de 20 ms entre dos decisiones del jugador. Un ciclo es la acción ESPERAR más todos los eventos hasta la siguiente decisión; el veredicto usa el peor de 30 ciclos:

Esbirros activos (= eventos pendientes)	Ciclo (mediana)	Peor de 30 ciclos	Veredicto
10	112 µs	115 µs	Cumple
100	1.05 ms	2.07 ms	Cumple
300	3.39 ms	4.63 ms	Cumple

Esbirros activos (= eventos pendientes)	Ciclo (mediana)	Peor de 30 ciclos	Veredicto
1 000 (prueba de estrés)	12.67 ms	13.23 ms	Cumple, no decide el veredicto

El ciclo cumple con 4 veces de margen hasta 300 esbirros y con 1.5 veces de margen incluso con 1 000. Ese resultado exigió ajustar el recolector de basura (umbrales 50 000, 20, 100, y congelar los objetos de larga vida tras el calentamiento): sin ajuste, el peor ciclo de la prueba de 1 000 esbirros llegaba a unos 55 ms. Con 50 veces más actores el ciclo seguiría creciendo en línea y es la primera medición que habría que repetir.

10. Resumen de las decisiones

Problema	Estructura elegida	Alternativa considerada
Agenda de eventos	Heap binario con posición por evento	Lista ordenada con búsqueda binaria
Caché de fichas	Cola doble LRU con contador de uso + archivo indexado	Tabla hash propia + lista doble
Historial de retroceso	Lista doble como pila con marcadores de acción	Instantáneas en buffer circular
Inventario	Lista doble con cursor	Arreglo contiguo con índice
Ordenamiento	Inserción + mezcla según tamaño y orden previo	Un solo algoritmo (quicksort o heapsort)
Guardado binario	Bloques con marcas + tabla de salas ordenada + pie fijo	Registros fijos con `struct`
Bitácora y reproducción	Lista doble en memoria + texto de solo anexado	Arreglo dinámico + log binario
Salas, mensajes y puntajes	Arreglo indexado, cola enlazada, registros fijos	Lista enlazada, buffer circular, texto (CSV/JSON)

11. Pendientes antes de la defensa

- Hecho: `--bench`, con la medición de cada sección y de la alternativa descartada. Falta repetir todo en Python 3.11+ y medir el ciclo de 1 000 esbirros con y sin el ajuste del recolector de basura.
- Hecho: el veneno y la regeneración del §2.10 (pulsos de veneno cada 80 unidades durante 400; regeneración de enemigos cada 200), con sus cambios en el historial de retroceso y sus eventos en la agenda.
- La selección del ordenamiento ya usa el tamaño y el orden previo (§4.9); el costo de comparar se mide pero no entra en la regla (sección 5); falta extenderlo a los índices del guardado y a las listas del catálogo.
- Hecho: comparación del guardado binario con JSON (§4.10, sección 6): gana en tamaño, en guardar y en traer una sala, y pierde al cargar la partida completa.
- Hecho: el módulo de ordenamiento vive ahora en la capa de lógica (§6).